rqbit ChunkTracker 및 진행 상황 추적 분석

개요

ChunkTracker는 rqbit에서 토렌트 다운로드 진행 상황을 추적하고 관리하는 핵심 구성 요소입니다. 이 시스템은 조각(piece) 및 청크(chunk) 단위로 다운로드 상태를 관리하며, 파일별 진행률, 우선순위 결정, 통계 생성 등의 기능을 제공합니다.

ChunkTracker 구조 분석

핵심 데이터 구조

```
pub  struct ChunkTracker {

// 다운로드 대기열 - 필요하지만 아직 요청하지 않은 조각들

queue_pieces: BF,

// 청크별 상태 - 파일에 쓰여진 청크들 (검증 전)

chunk_status: BF,

// 완전히 다운로드되고 검증된 조각들

have: BoxBitV,

// 사용자가 선택한 조각들

selected: BF,

// 파일별 다운로드된 바이트 수

per_file_bytes: Vec<u64>,

// 토렌트 길이 정보
```

```
lengths: Lengths,

// 빠른 통계 조회를 위한 캐시

hns: HaveNeededSelected,

}
```

비트벡터 시스템

1. queue_pieces (대기열 조각)

- **목적**: 다운로드가 필요한 조각들을 추적
- **초기값**: `selected & !have` (선택되었지만 보유하지 않은 조각)
- **동작**: 피어에게 요청하는 순간 0으로 설정 (중복 요청 방지)

2. chunk_status (청크 상태)

- **목적**: 파일에 쓰여진 청크들을 추적 (검증 전)
- **단위**: 조각이 아닌 청크 단위로 관리
- **용도**: 조각 완료 여부 판단 및 부분 다운로드 추적

3. have (보유 조각)

- **목적**: 완전히 다운로드되고 해시 검증된 조각들
- **타입**: `BoxBitV` (동적 크기 비트벡터)
- **신뢰성**: 가장 신뢰할 수 있는 상태 정보

4. selected (선택된 조각)

- **목적**: 사용자가 선택한 파일에 해당하는 조각들
- **변경**: `update_only_files` 호출 시에만 변경

- **영향**: 다운로드 우선순위와 통계 계산에 사용

HaveNeededSelected 통계 구조

구조체 정의

```
#[derive(Default, Debug, PartialEq, Eq, Clone, Copy)]
pub struct HaveNeededSelected {
pub have_bytes: u64, // 다운로드되고 검증된 바이트
pub needed_bytes: u64, // 아직 필요한 바이트
pub selected_bytes: u64, // 선택된 총 바이트
}
```

핵심 메서드

```
impl HaveNeededSelected {
// 현재 진행률 (다운로드된 바이트)
pub const fn progress(&self) -> u64 {
self.selected_bytes - self.needed_bytes
}
// 총 크기
pub const fn total(&self) -> u64 {
self.selected_bytes
}
```

```
// 완료 여부
pub const fn finished(&self) -> bool {
self.needed_bytes == 0
}
}
```

조각 상태 관리

조각 마킹 결과

```
#[derive(Debug)]
pub enum ChunkMarkingResult {
PreviouslyCompleted, // 이미 완료된 조각
NotCompleted, // 아직 완료되지 않음
Completed, // 방금 완료됨
}
```

청크 다운로드 마킹

```
pub fn mark_chunk_downloaded(&mut self, piece: &Piece<ByteBuf<'_>>) -> Option<Ch
let chunk_info = self.lengths.chunk_info_from_received_data(
self.lengths.validate_piece_index(piece.index)?,
piece.begin,
piece.len().try_into().unwrap(),
```

```
)?;
let chunk_range = self.lengths.chunk_range(chunk_info.piece_index);
let chunk_range = self.chunk_status.get_mut(chunk_range).unwrap();
if chunk_range.all() {
return Some(ChunkMarkingResult::PreviouslyCompleted);
}
chunk_range.set(chunk_info.chunk_index as usize, true);
if chunk_range.all() {
return Some(ChunkMarkingResult::Completed);
}
Some(ChunkMarkingResult::NotCompleted)
}
```

과정:

1. 수신된 데이터에서 청크 정보 추출
2. 해당 조각의 청크 범위 획득
3. 이미 완료된 조각인지 확인
4. 청크를 완료로 마킹
5. 조각 전체가 완료되었는지 확인

조각 완료 처리

```
pub fn mark_piece_downloaded(&mut self, idx: ValidPieceIndex) {
```

```
let id = idx.get() as usize;
if !self.have.as_slice()[id] {
self.have.as_slice_mut().set(id, true);
let len = self.lengths.piece_length(idx) as u64;
self.hns.have_bytes += len;
if self.selected[id] {
self.hns.needed_bytes -= len;
}
}
}
```

특징:

- 중복 마킹 방지
- 통계 자동 업데이트
- 선택된 조각만 needed_bytes에서 차감

우선순위 및 대기열 관리

대기열 조각 반복자

```
pub(crate) fn iter_queued_pieces<'a>(
&'a self,
file_priorities: &'a FilePriorities,
file_infos: &'a FileInfos,
```

```
) -> impl Iterator<Item = ValidPieceIndex> + 'a {
file_priorities
.iter()
.filter_map(|p| Some((*p, file_infos.get(*p)?)))
.filter(|(id, f)| self.per_file_bytes[*id] != f.len) // 완료되지 않은 파일
.flat_map(|(_id, f)| f.iter_piece_priorities()) // 파일의 조각들
.filter(|id| self.queue_pieces[*id]) // 대기열에 있는 조각
.filter_map(|id| id.try_into().ok())
.filter_map(|id| self.lengths.validate_piece_index(id))
}
```

우선순위 결정 과정:

1. 파일 우선순위 순서대로 처리
2. 완료되지 않은 파일만 선택
3. 파일 내 조각 우선순위 적용
4. 대기열에 있는 조각만 필터링
5. 유효한 조각 인덱스로 변환

조각 예약

```
pub fn reserve_needed_piece(&mut self, index: ValidPieceIndex) {
self.queue_pieces.set(index.get() as usize, false)
}
```

목적:

- 피어에게 요청한 조각을 대기열에서 제거
- 중복 요청 방지
- 다른 피어가 같은 조각을 요청하지 않도록 함

파일별 진행 상황 추적

파일별 바이트 계산

```
fn recalculate_per_file_bytes(&mut self, file_infos: &FileInfos) {
for (slot, fi) in self.per_file_bytes.iter_mut().zip(file_infos.iter()) {
*slot = fi
.piece_range
.clone()
.filter(|p| self.have.as_slice()[*p as usize])
.map(|id| {
self.lengths.size_of_piece_in_file(id, fi.offset_in_torrent, fi.len)
})
.sum();
}
}
```

계산 과정:

1. 파일의 조각 범위 순회
2. 보유한 조각만 필터링

3. 각 조각이 파일에 기여하는 바이트 수 계산

4. 총합 계산

조각 완료 시 파일 업데이트

```
pub fn update_file_have_on_piece_completed(
&mut self,
piece_id: ValidPieceIndex,
file_id: usize,
file_info: &FileInfo,
) -> u64 {
let diff_have = self.lengths.size_of_piece_in_file(
piece_id.get(),
file_info.offset_in_torrent,
file_info.len,
);
self.per_file_bytes[file_id] += diff_have;
file_info.len.saturating_sub(self.per_file_bytes[file_id])
}
```

반환값: 파일의 남은 바이트 수

파일 선택 업데이트

update_only_files 메서드

```
pub fn update_only_files(
&mut self,
file_infos: &FileInfos,
new_only_files: &HashSet<usize>,
) -> anyhow::Result<HaveNeededSelected> {
let selected = compute_selected_pieces(
&self.lengths,
|idx| new_only_files.contains(&idx),
file_infos,
);
let prev_selected = std::mem::replace(&mut self.selected, selected);


// 새로 선택된 조각들을 대기열에 추가
{
let mut b = BF::from_boxed_slice(
vec![0u8; self.lengths.piece_bitfield_bytes()].into_boxed_slice(),
);
for idx in self
.selected
.iter_ones()
.filter(|idx| !prev_selected[*idx] && !self.have.as_slice()[*idx])
{
```

```
b.set(idx, true);
}

for idx in b.iter_ones() {
if let Some(idx) = self.lengths.validate_piece_index(idx as u32) {
self.mark_piece_broken_if_not_have(idx);
}
}
}

// 선택 해제된 조각들을 대기열에서 제거
{
let mut have_or_selected: BF = self.selected.clone();
have_or_selected |= self.have.as_slice();
self.queue_pieces &= have_or_selected;
}

self.hns = self.calc_hns();
Ok(self.hns)
}
```

과정:

1. 새로운 선택 조각 계산

2. 새로 선택된 조각들을 대기열에 추가

3. 선택 해제된 조각들을 대기열에서 제거

4. 통계 재계산

통계 시스템 통합

세션 레벨 통계

```
gen_stats!(SessionCountersAtomic SessionCountersSnapshot, [

fetched_bytes u64,

uploaded_bytes u64,

blocked_incoming u64,

blocked_outgoing u64

], []);
```

```
pub struct SessionStats {

pub counters: SessionCountersAtomic,

pub peers: Arc<AggregatePeerStatsAtomic>,

pub down_speed_estimator: SpeedEstimator,

pub up_speed_estimator: SpeedEstimator,

pub startup_time: Instant,

}
```

토렌트 레벨 통계

```rust
#[derive(Serialize, Debug)]
pub struct TorrentStats {
pub state: TorrentStatsState,
pub file_progress: Vec<u64>,
pub error: Option<String>,
pub progress_bytes: u64,
pub uploaded_bytes: u64,
pub total_bytes: u64,
pub finished: bool,
pub live: Option<LiveStats>,
}
```

실시간 통계

```rust
#[derive(Serialize, Default, Debug)]
pub struct LiveStats {
pub snapshot: StatsSnapshot,
pub average_piece_download_time: Option<Duration>,
pub download_speed: Speed,
pub upload_speed: Speed,
pub time_remaining: Option<DurationWithHumanReadable>,
}
```

속도 추정 및 ETA 계산

SpeedEstimator 사용

```
impl Session {
pub(crate) fn start_speed_estimator_updater(self: &Arc<Self>) {
self.spawn(
debug_span!(parent: self.rs(), "speed_estimator"),
"speed_estimator",
{
let s = Arc::downgrade(self);
async move {
let mut i = tokio::time::interval(Duration::from_secs(1));
loop {
i.tick().await;
let s = s.upgrade().context("session is dead")?;
let now = Instant::now();
let fetched = s.stats.counters.fetched_bytes.load(Ordering::Relaxed);
let uploaded = s.stats.counters.uploaded_bytes.load(Ordering::Relaxed);
s.stats.down_speed_estimator.add_snapshot(fetched, None, now);
s.stats.up_speed_estimator.add_snapshot(uploaded, None, now);
}
}
```

```
},
)
}
}
```

특징:

- 1초마다 속도 업데이트
- 5개 샘플 기반 이동 평균
- ETA 계산 지원

오류 처리 및 복구

손상된 조각 처리

```
pub fn mark_piece_broken_if_not_have(&mut self, index: ValidPieceIndex) {
if self.have.as_slice().get(index.get() as usize).map(|r| *r).unwrap_or_default(
return;
}
debug!("marking piece={} as broken", index);
self.queue_pieces.set(index.get() as usize, true);
if let Some(s) = self.chunk_status.get_mut(self.lengths.chunk_range(index)) {
s.fill(false);
}
```

```
}
```

복구 과정:

1. 이미 보유한 조각은 건드리지 않음
2. 대기열에 다시 추가
3. 청크 상태 초기화
4. 재다운로드 대상으로 마킹

성능 최적화

1. 비트벡터 연산

- 효율적인 비트 조작을 통한 빠른 상태 확인
- 메모리 효율적인 상태 저장
- 비트 단위 논리 연산 활용

2. 캐시된 통계

- `HaveNeededSelected` 구조체를 통한 빠른 통계 조회
- 상태 변경 시에만 재계산
- 자주 사용되는 값들의 사전 계산

3. 파일별 바이트 추적

- 조각 완료 시 증분 업데이트
- 전체 재계산 최소화
- O(1) 파일 진행률 조회

동시성 고려사항

스레드 안전성

- ChunkTracker 자체는 `Send` + `Sync` 가 아님
- 상위 레벨에서 적절한 동기화 필요
- 원자적 연산을 통한 통계 업데이트

상태 일관성

- 여러 비트벡터 간의 일관성 보장
- 상태 변경 시 관련된 모든 필드 업데이트
- 트랜잭션적 상태 변경

결론

ChunkTracker는 rqbit의 진행 상황 추적 시스템의 핵심으로서 다음과 같은 기능을 제공합니다:

1. **정확한 진행률 추적**: 조각 및 청크 단위의 세밀한 상태 관리
2. **효율적인 우선순위 관리**: 파일 우선순위 기반 다운로드 순서 결정
3. **실시간 통계**: 속도, ETA, 완료율 등의 실시간 계산
4. **유연한 파일 선택**: 동적 파일 선택 변경 지원
5. **오류 복구**: 손상된 조각 감지 및 재다운로드

이러한 설계를 통해 rqbit은 사용자에게 정확하고 유용한 다운로드 진행 정보를 제공하며, 효율적인 대역폭 활용과 빠른 다운로드 완료를 달성합니다.